

Consignas del Proyecto

Agente Autónomo de Comparación de Contratos - LegalMove

Contexto y objetivos

Imagina que trabajas como AI Engineer en una empresa de tecnología legal (LegalMove) que procesa miles de enmiendas de contratos cada mes.

Actualmente, el equipo de Compliance pasa más de 40 horas a la semana comparando manualmente contratos originales con sus enmiendas para identificar cambios, evaluar impacto legal y derivar documentos para revisión. Este proceso manual es lento, propenso a errores humanos y limita la escalabilidad del negocio.

La misión es construir un Agente Autónomo de Comparación de Contratos. El sistema recibirá imágenes escaneadas del contrato original y de su adenda, las leerá con IA de visión y usará agentes de IA para entender el contexto y extraer las cláusulas modificadas, devolviendo un reporte estructurado y procesable automáticamente.

Objetivo

Desarrollar un sistema multi-agente autónomo que procese imágenes escaneadas de un contrato original y su adenda. El sistema deberá extraer el texto utilizando modelos fundacionales multimodales y, mediante dos agentes especializados, identificar, extraer y resumir los cambios legales aplicados. El resultado final debe ser un JSON estrictamente validado, con trazabilidad completa de cada paso del proceso.

Stack técnico requerido

- OpenAI GPT-4o (Vision): parseo de imágenes de contratos y conversión a texto estructurado.
- LangChain: implementación y orquestación de dos agentes colaborativos.
- Pydantic: validación y estructuración del output final.
- Langfuse: trazabilidad completa del workflow, incluyendo image parsing, agentes y validación.
- Python + python-dotenv: lenguaje base y manejo seguro de variables de entorno.

Consigna resumen

Crear una aplicación en Python que reciba dos imágenes: contrato original y adenda. El sistema deberá usar un LLM multimodal para leer los documentos y pasarlos por un flujo de dos agentes: uno que entiende el contexto y estructura del documento, y otro que extrae las diferencias. La salida debe ser un JSON validado por Pydantic con las secciones modificadas, los temas legales afectados y un resumen preciso de los cambios. Todas las llamadas a la API y acciones de los agentes deben quedar registradas en un dashboard de Langfuse.

Consigna técnica

Paso 1 - Parsing multimodal de imágenes

Implementar una función `parse_contract_image()` que reciba el path de una imagen JPEG/PNG, la codifique en base64 y realice una llamada a la API de GPT-4o con capacidad multimodal. El prompt de visión debe instruir al modelo a extraer el texto completo del contrato de la forma más fiel posible. Este paso debe ejecutarse dos veces: una para el contrato original y otra para la enmienda. La observabilidad de estas ejecuciones debe registrarse mediante spans de Langfuse dentro del pipeline principal.

Paso 2 - Agente 1: Contextualización

Implementar `ContextualizationAgent`. Su responsabilidad es recibir los dos textos parseados y producir un análisis de estructura comparada: identificar qué secciones existen en ambos documentos, cómo se corresponden entre sí y cuál es el propósito general de cada bloque. El output puede ser texto estructurado que funcione como mapa contextual para el siguiente agente. Este agente no extrae cambios; solo construye el contexto necesario para el análisis posterior.

Paso 3 - Agente 2: Extracción de cambios

Implementar `ExtractionAgent`. Recibe como input el output del Agente 1 junto con ambos textos, y tiene la responsabilidad exclusiva de identificar, aislar y describir cada cambio introducido por la enmienda. Debe distinguir entre adiciones, eliminaciones y modificaciones. Su output debe ser un JSON estructurado con los tres campos requeridos, listo para ser validado por Pydantic.

Paso 4 - Validación con Pydantic

Definir el modelo `ContractChangeOutput` con los campos requeridos: `sections_changed: List[str]`, `topics_touched: List[str]` y `summary_of_the_change: str`. El output del Agente 2 debe cumplir este schema y ser validado utilizando Pydantic, ya sea con `model_validate()` o mediante structured outputs usando `response_format=ContractChangeOutput`.

Paso 5 - Trazabilidad con Langfuse

Instrumentar el pipeline con trazabilidad usando Langfuse. Se recomienda implementar un span raíz que represente la ejecución completa del pipeline y spans hijos para cada etapa principal. Cada span debe incluir input, output, latencia y metadata útil para debugging o auditoría.

Estructura sugerida de trazas Langfuse

```
contract-analysis
- parse_original_contract
- parse_amendment_contract
- contextualization_agent
- extraction_agent
```

Modelo de salida requerido

```
sections_changed: List[str] - identificadores de secciones modificadas.
topics_touched: List[str] - categorías legales o comerciales afectadas.
summary_of_the_change: str - descripción detallada de los cambios.
```

Entregables del proyecto y requisitos de entrega

El proyecto se entrega mediante un repositorio público de GitHub que contiene los siguientes elementos:

Entregable	Archivo	Descripción
Script principal	src/main.py	Entry point que acepta dos paths de imágenes como argumentos y ejecuta el pipeline completo.
Agente 1	src/agents/contextualization_agent.py	Agente de contextualización con system prompt y lógica propios.
Agente 2	src/agents/extraction_agent.py	Agente de extracción con system prompt y lógica propios.
Utilidades de imagen	src/image_parser.py	Funciones de validación, encoding y llamadas multimodales.
Modelos Pydantic	src/models.py	Modelo ContractChangeOutput con los tres campos requeridos.
Imágenes de prueba	data/test_contracts/	Mínimo 2 pares de contratos (4 imágenes) con README explicativo.
README	README.md	Documentación completa con diagramas, arquitectura, setup, uso y decisiones técnicas.
Dependencias	requirements.txt + .env.example	Dependencias con versiones fijadas y template de variables de entorno.
Integración Langfuse	Instrumentación del pipeline principal	Traza que agrupe las etapas del análisis con jerarquía de spans para cada etapa del pipeline.

Defensa en vivo - instancia final

En lugar de un video pregrabado, se realizará una sesión en vivo de presentación y defensa técnica del proyecto durante 30 minutos.

Durante la sesión, se deberá:

- Presentar brevemente el contexto de la empresa LegalMove y el problema que el sistema resuelve.
- Mostrar el pipeline en funcionamiento con al menos dos pares de contratos: un caso con cambios simples y uno con cambios complejos.
- Explicar en el dashboard de Langfuse cómo se ve la traza completa de una ejecución: spans, inputs/outputs de cada agente y métricas de tokens y latencia.
- Responder preguntas del corrector sobre decisiones técnicas: por qué dos agentes en lugar de uno, por qué GPT-4o para el parsing, cómo se diseñaron los system prompts y cómo se maneja la validación de errores.

Recursos adicionales

Contratos de prueba simulados

Para los pares de contratos de prueba se pueden utilizar los archivos ya generados en los recursos adicionales o generar documentos PDF simples con herramientas como Canva, Google Docs o Word, exportándolos luego como imágenes.

Escenarios sugeridos

- Par 1 - Cambios simples: contrato de servicios donde la enmienda modifica únicamente el monto mensual y la fecha de vencimiento.
- Par 2 - Cambios complejos: contrato de confidencialidad donde la enmienda agrega una cláusula nueva, modifica el alcance territorial y elimina una restricción de uso.

OpenAI Vision

Consultar la documentación oficial: [Images and vision | OpenAI API](#).

Setup de Langfuse

- Crear una cuenta en cloud.langfuse.com.
- Crear un nuevo proyecto y obtener `LANGFUSE_PUBLIC_KEY` y `LANGFUSE_SECRET_KEY`.
- Instalar con `pip install langfuse`.
- Inicializar el cliente con las keys desde variables de entorno.

Variables de entorno requeridas (.env.example)

```
OPENAI_API_KEY=your-key-here
LANGFUSE_PUBLIC_KEY=pk-lf-xxx
LANGFUSE_SECRET_KEY=sk-lf-xxx
LANGFUSE_HOST=https://cloud.langfuse.com
```